Vyshnavi Parisha

2403A51L34

Batch-52

ASSIGNMENT 16.2

Database Design and Queries: Schema Design and SQL Generation

Task Descrption-1: (Design Database Schema for Student Management System)

PROMPT: Generate SQL CREATE TABLE statements for a Student Management System with tables Students, Courses, and Enrollments including primary keys and foreign keys while maintaining normalization

CODE:

OUTPUT:

Explanation :

- The database schema contains three tables: **Students, Courses, and Enrollments**.
- The **Students table** stores student details such as ID, name, and email.
- The **Courses table** stores course information including course ID, course name, and credits.
- The **Enrollments table** connects students and courses using foreign keys, allowing many students to enroll in many courses
- This design follows **1NF, 2NF, and 3NF normalization rules** by removing redundant data and ensuring proper relationships between tables.

Task Description- 2: (Insert Sample Records)

PROMPT : Generate SQL INSERT statements to populate Students, Courses, and Enrollments tables with sample data.

CODE:

OUTPUT:

Explanation :

- These SQL statements insert sample data into the tables.
- Three students and four courses are added to their respective tables.
- The **Enrollments table** records which students are registered in which courses. This data helps test queries and validate database relationships.

Task Description- 3: (Perform CRUD Operations)

Prompt : Generate SQL queries for CRUD operations including retrieving courses, updating student email, and deleting an enrollment record.

Code :

Output :

Explanation :

- CRUD operations represent **Create, Read, Update, and Delete** database actions.
- The **SELECT query** retrieves all courses from the database.
- The **UPDATE query** modifies a student's email address.
- The **DELETE query** removes an enrollment record from the database.

Task Description- 4: (Execute Aggregate Queries)

Prompt : Write SQL aggregate queries to count the number of students enrolled in each course and display courses with more than one student.

Code :

Output :

Explanation :

- Aggregate queries summarize data using functions such as **COUNT()**.
- The first query counts the number of students enrolled in each course.
- The second query uses the **HAVING clause** to filter courses that have more than one enrolled student.

Task Description- 5: (Query Optimization)

Prompt : Optimize SQL queries by replacing subqueries with JOIN operations to improve performance

Code :

Output :

Explanation :

- The original query uses a **subquery** to find students enrolled in a course.
- The optimized query uses a **JOIN**, which is generally faster because it directly connects the tables during execution.
- Query optimization helps improve performance, especially when working with large datasets.